student_dashboard.js - Complete Documentation

File Location: `src/js/student_dashboard.js`

Purpose: Student dashboard interactivity for managing personal event activities

Dependencies: `modal1.js`, `history_handler.js`

Used by: `student_homepage.html`

Overview

The `student_dashboard.js` file implements the student management interface. It handles displaying student-specific data: registered events, event requests, complaints, and feedback. Similar structure to `admin_dashboard.js` but with student-focused operations.

Module Variables

currentListType

```
let currentListType = '';
```

Purpose: Track which student list is currently displayed.

currentListData

```
let currentListData = [];
```

Purpose: Store the fetched list data for rendering.

Core Functions

openStudentListModal(type)

Purpose: Opens modal displaying different student lists.

Parameters:

- `type` (string) - List type to display. Possible values:
  - `'my_registrations'` - Events student registered for
  - `'my_event_requests'` - Events student requested
  - `'my_complaints'` - Complaints student submitted
  - `'my_feedback'` - Feedback student submitted
  - `'participated'` - Events student participated in (history)
  - `'past_participated'` - Past events attended

Returns: void

What it does:

1. Sets `currentListType` to requested type
2. Opens `student_list_modal.html`
3. Calls `loadStudentListData(type)`

Code:

```
function openStudentListModal(type) {
    currentListType = type;

    openModal('../../frontends/student/student_list_modal.html', 'studentListModal', function(modal) {
        loadStudentListData(type);
    });
}
```

Example Usage:

```
// Student clicks "My Registrations"
openStudentListModal('my_registrations');

// Student clicks "My Complaints"
openStudentListModal('my_complaints');
```

---

loadStudentListData(type)

Purpose: Fetches student-specific data based on list type.

Parameters:

- `type` (string) - List type to load

Returns: void (async)

What it does:

1. Shows loading message
2. Determines appropriate PHP endpoint based on type
3. Fetches data from endpoint
4. Stores in `currentListData`
5. Calls `renderStudentList()`

Endpoint Mapping:

```
switch(type) {
    case 'my_registrations':
        url = '../../php/student/get_my_registrations.php';
        break;
    case 'my_event_requests':
        url = '../../php/student/get_my_event_requests.php';
        break;
    case 'my_complaints':
        url = '../../php/student/get_my_complaints.php';
        break;
    case 'my_feedback':
        url = '../../php/student/get_my_feedback.php';
        break;
    case 'participated':
    case 'past_participated':
        url = '../../php/student/get_participated_events.php';
        break;
}
```

Code:

```
async function loadStudentListData(type) {
    const contentDiv = document.getElementById('studentListContent');
    if (!contentDiv) return;

    contentDiv.innerHTML = '<p style="text-align:center; padding:40px;">Loading your data...</p>';

    let url = '';
    // ... determine URL from type

    try {
        const res = await fetch(url);
        if (!res.ok) throw new Error('Server error');

        const data = await res.json();

        currentListData = Array.isArray(data) ? data : [];

        if (currentListData.length === 0) {
            contentDiv.innerHTML = '<p style="text-align:center; padding:40px;">You have no items yet.</p>';
        } else {
            renderStudentList(currentListData, type);
        }
    } catch (err) {
        console.error(err);
        contentDiv.innerHTML = `<p style="color:red; text-align:center;">Failed to load data.<br><small>${err.message}</small></p>`;
    }
}
```

---

renderStudentList(data, type)

Purpose: Renders appropriate HTML for different student list types.

Parameters:

- `data` (array) - Array of items to render
- `type` (string) - List type (determines rendering)

Returns: void

What it does:

1. Sets modal title based on type
2. Renders type-specific HTML structure
3. Creates list items with student-appropriate actions
4. Shows view/details buttons

Title Mapping:

```
const titles = {
    'my_registrations': 'My Registrations',
    'my_event_requests': 'My Event Requests',
    'my_complaints': 'My Complaints',
    'my_feedback': 'My Feedback',
    'participated': 'Participated Events',
    'past_participated': 'Past Events'
};
```

Rendering by Type:

My Registrations

```
case 'my_registrations':
    infoHTML = `
        <div class="list-info">
            <h4>${item.event_name}</h4>
            <p>${item.date} | Status: ${item.status}</p>
        </div>`;
    buttonText = 'View Event';
    onclick = `viewEventDetailOnly(${item.event_id})`;
    break;
```

Rendered Output:

```
┌──────────────────────────────────┐
│ Sports Day 2024                  │
│ 2024-05-15 | Status: Confirmed   │
│                          [View] │
└──────────────────────────────────┘
```

My Event Requests

```
case 'my_event_requests':
    infoHTML = `
        <div class="list-info">
            <h4>${item.event_name}</h4>
            <p>Status: <strong>${item.status}</strong> | Requested: ${item.request_date}</p>
        </div>`;
    buttonText = 'View Details';
    onclick = `openHistoryDetail('eventRequest', ${item.request_id})`;
    break;
```

Status Indicators:

- □ `pending` - Waiting for admin approval
- □ `approved` - Event approved, created
- □ `rejected` - Admin rejected request

My Complaints

```
case 'my_complaints':
    const replyCount = parseInt(item.reply_count) || 0;
    const hasReply = replyCount > 0;
    infoHTML = `
        <div class="list-info">
            <h4>${item.event_name}</h4>
            <p>Submitted: ${item.complaint_date} | ${hasReply ? '□ Admin replied' : '□ Pending'}</p>
        </div>`;
    buttonText = 'View Details';
    onclick = `openHistoryDetail('complaint', ${item.complaint_id})`;
    break;
```

My Feedback

```
case 'my_feedback':
    infoHTML = `
        <div class="list-info">
            <h4>${item.event_name}</h4>
            <p>Rating: ${'□'.repeat(parseInt(item.rating) || 0)} | ${item.feedback_date}</p>
        </div>`;
    buttonText = 'View Feedback';
    onclick = `openHistoryDetail('feedback', ${item.feedback_id})`;
    break;
```

Participated Events

```
case 'participated':
case 'past_participated':
    infoHTML = `
        <div class="list-info">
            <h4>${item.event_name}</h4>
            <p>Participated: ${item.date} | ${item.category}</p>
        </div>`;
    buttonText = 'View Event';
    onclick = `viewEventDetailOnly(${item.event_id})`;
    break;
```

Student Action Functions

viewEventDetailOnly(eventId)

Purpose: Opens event details modal (read-only for students).

Parameters:

- `eventId` (integer) - Event to view

What it does:

1. Fetches event from `get_event_details.php`
2. Opens event details modal
3. Shows all event information
4. Disables registration buttons (already registered)

Differences from Admin:

- Students can only view, not edit
- No admin-specific fields shown
- Can submit feedback if event completed
- Can view other student feedback

Integration Points

With modal1.js

```
// Uses generic modal system
openModal('../../frontends/student/student_list_modal.html', 'studentListModal', function() {
    loadStudentListData(type);
});
```

With history_handler.js

```
// Opens specific complaint/request details
openHistoryDetail('complaint', complaintId);
openHistoryDetail('eventRequest', requestId);
openHistoryDetail('feedback', feedbackId);
```

Modal Structure

student_list_modal.html Expected Structure

```
<div class="modal-content">
    <div class="modal-header">
        <h2 id="listModalTitle">Student List</h2>
        <span class="close" onclick="closeModal('studentListModal')">×</span>
    </div>

    <div id="studentListContent">
        <!-- List items rendered here -->
    </div>
</div>
```

Generated List Item Structure

```
<div class="list-item">
    <div class="list-info">
        <h4>Item Title</h4>
        <p>Metadata info</p>
    </div>
    <button class="action-btn" onclick="...">View/Details</button>
</div>
```

Data Flow Example

Complete Workflow: Viewing Complaints

```
Student clicks "My Complaints"
    ↓
openStudentListModal('my_complaints')
    ↓
Sets currentListType = 'my_complaints'
    ↓
Opens student_list_modal.html
    ↓
loadStudentListData('my_complaints') called
    ↓
Fetches get_my_complaints.php
    ↓
Response: Array of student's complaints
    ↓
Stored in currentListData
    ↓
renderStudentList(data, 'my_complaints')
    ↓
Generates list items showing:
    Event Name
    Submit date | Admin reply status
    [View Details] button
    ↓
Student sees their complaints
    ↓
Clicks complaint
    ↓
openHistoryDetail('complaint', complaintId)
    ↓
Shows complaint detail with admin reply
```

Error States

No Items

```
<p style="text-align:center; padding:40px;">You have no items yet.</p>
```

Load Error

```
<p style="color:red; text-align:center;">Failed to load data.<br><small>Error message</small></p>
```

Loading State

```
<p style="text-align:center; padding:40px;">Loading your data...</p>
```

Performance Considerations

□ Current Strengths

- Lazy loading (data fetched on demand)
- Efficient rendering with `.join('')`
- Reusable modal pattern

⚠ Potential Improvements

1. Search Within Lists

```
function searchStudentList(searchTerm) {
    const filtered = currentListData.filter(item =>
        item.event_name?.includes(searchTerm)
    );
    renderStudentList(filtered, currentListType);
}
```

2. Filter by Status

```
function filterByStatus(status) {
    const filtered = currentListData.filter(item =>
        item.status === status
    );
    renderStudentList(filtered, currentListType);
}
```

3. Pagination for Large Lists

```
async function loadStudentListData(type, page = 1) {
    const limit = 10;
    const offset = (page - 1) * limit;
    // ...
}
```

Summary

student_dashboard.js provides:

Feature	Purpose
openStudentListModal()	Open list modal for student data
loadStudentListData()	Fetch student-specific data
renderStudentList()	Render type-specific HTML
viewEventDetailOnly()	Display event details

Creates a student-focused interface for managing personal activities.

Last Updated: May 22, 2026

File Size: ~120 lines of code

Complexity: Medium

Criticality: High (core student feature)